Sorting Race Results with Two Algorithms

Assignment 1 | Analysis of Algorithms | 5 points

Purpose

A race organiser needs to turn thousands of recorded finish times into a reliable leaderboard. In this assignment, you will implement two sorting algorithms from the course, check that both produce the same correct result, and use measured evidence to explain how their performance changes as the data grows. The work applies algorithm design, operation counting, asymptotic analysis, and empirical evaluation.

The challenge

Build a small race-results sorting tool. Each record is a non-negative finish time in milliseconds. Sort the records from fastest to slowest (ascending time) using both Merge Sort and LSD Radix Sort. Your program must preserve the original input so that each algorithm receives the same data.

Use the same programming language for both implementations. Java, Python, C++, or Go are acceptable. Do not call a built-in sorting function inside either algorithm. You may use built-in facilities to generate data, copy arrays, measure elapsed time, and produce charts.

1 Implement and verify

Implement Merge Sort and a stable least-significant-digit Radix Sort in base 10. Radix Sort must correctly process every value in the specified range, including values with leading zeroes in their decimal representation and repeated finish times.

Before benchmarking, test both algorithms on a short list you can check by hand. Include an empty list, a one-value list, repeated values, already sorted values, and reverse-ordered values. For every generated test, verify that each output is in ascending order and contains exactly the same values as its input. Report how you checked correctness.

2 Create the race data

For each input size, generate one random dataset of integer finish times from 0 through 9,999,999 milliseconds. Test these five sizes: 1,000; 5,000; 10,000; 50,000; and 100,000 records. Repeated times are allowed. Record the random seed or save each generated dataset so your results can be reproduced. Use the same input for both algorithms at each size. The already ordered, reverse-ordered, and repeated-value lists from the correctness checks do not need separate performance measurements.

3 Measure performance

For each algorithm and input size, record elapsed time and a count of basic operations. Run each measurement three times on the same machine and report the median time. Exclude data generation, output printing, and chart creation from the timed section. State your computer, programming language, and how elapsed time was measured.

Choose an operation count that matches each implementation and define it clearly. For example, count key comparisons and element copies for Merge Sort, and digit inspections and element placements for Radix Sort. Because the algorithms perform different kinds of work, show their operation counts separately and do not treat the two counts as directly equivalent.

4 Analyse what the results mean

Present a clearly labelled results table and charts of elapsed time and operation counts against input size. Label the algorithms clearly and include units.

For each algorithm, state and justify its best-case, average-case, and worst-case asymptotic running time using the course materials. For Radix Sort, identify the number of decimal digits and the role of the key range in interpreting the result. Compare the theoretical growth with your measured curves. Explain what the operation counts reveal and one practical trade-off you observed. Do not claim that one algorithm is always faster based on one machine or one dataset.

Finish with a reflection of no more than 300 words: What did implementing and measuring the algorithms teach you that reading their complexity alone did not? Support your answer with at least one result from your experiment.

Submission

Submit exactly one DOCX or PDF file through MS Teams by the announced deadline. This single document is the complete submission. Do not submit ZIP archives, separate code files, external cloud links, or supplementary files. Include all of the following in the document:

1. The report: method, computer and programming-language details, correctness checks, results table, embedded charts, analysis, and reflection.
2. Complete source code for Merge Sort, Radix Sort, and the test and benchmark driver. Add it as selectable text in a clearly labelled appendix and preserve indentation with a monospaced font. Do not use screenshots of code.
3. Test evidence showing that both algorithms produce correct results, plus the measurement results used in the charts. Include the random seed or describe how the data can be reproduced; raw generated datasets do not need to be printed in full.
4. Embedded charts and tables. Make sure they are readable in both Word and PDF and do not depend on external links.

Keep the report, code, test evidence, and results together in the one file you upload to MS Teams. Late work and collaboration are handled under the course assessment and academic integrity rules. AI tools may be used for syntax questions or debugging; the implementations, measurements, analysis, and reflection must be your own work.

Marking rubric

COMPONENT	Points	What successful work shows
CORRECT IMPLEMENTATION AND VERIFICATION	2.0	The included source code implements both algorithms without built-in sorting; test evidence checks order and preserves all input values.
DATA COLLECTION AND VISUALIZATION	1.0	The single-file report includes reproducible measurements, a results table, and readable, labelled charts.
ANALYSIS AND DISCUSSION	1.5	Complexity claims are justified; measured trends and operation counts are interpreted accurately.
CODE CLARITY AND STRUCTURE	0.5	Code is included as readable text, is modular, and uses clear names and helpful comments.